

Pearls AQI Predictor

An End-to-End Serverless Air Quality Forecasting System for Karachi, Pakistan

Class Project Report

Gaurav Khatwani

Repository: github.com/khgaurav-del/aqi-predictor-karachi

August 2026

Table of Contents

1. Project Overview	3
2. System Architecture	4
3. Data Collection and Feature Engineering	5
4. Modeling and Evaluation	6
5. Model Explainability (SHAP)	7
6. Automation and Deployment	9
7. Challenges Along the Way	11
8. What Could Come Next	12
9. Conclusion	13

1. Project Overview

This project forecasts Karachi's air quality three days in advance. It runs on its own: a script collects fresh weather and pollution data every hour, another script retrains the prediction model every day, and a dashboard shows the current forecast. Nobody has to log in and run anything by hand once it is set up.

It focuses on Karachi (24.8607 N, 67.0011 E) and reports air quality using the standard US EPA AQI scale (0 to 500), calculated from PM2.5 levels in the air.

What this project set out to do

- Collect pollution and weather data for Karachi automatically, hour after hour
- Turn that raw data into useful features: recent trends, rolling averages, time-of-day patterns
- Train a few different models and see which one actually predicts AQI three days out
- Automate the whole thing so it keeps working without anyone watching it
- Show the forecast on a simple dashboard, with a warning when air quality turns hazardous

What it is built with

Component	Technology	Purpose
Pollutant data	OpenWeather Air Pollution API	PM2.5, PM10, CO, NO, NO2, O3, SO2, NH3
Weather data	Open-Meteo Archive API	Temperature, humidity, wind, pressure, cloud cover
Feature store	MongoDB Atlas (M0, free tier)	Stores every hourly reading so models can be trained on it
Automation	GitHub Actions	Runs the hourly collection and daily retraining, no server needed
Modeling	scikit-learn, XGBoost	Ridge, Random Forest, and XGBoost regression
Explainability	SHAP	Shows which features actually drive the model's predictions
Dashboard	Streamlit	Displays the live forecast and hazard alerts

2. System Architecture

The system is built in four stages. Each one runs on its own and logs what it did, so it is easy to check whether something worked.

- Feature Pipeline (hourly): grabs the latest pollution and weather readings, works out the derived features using recent history, and saves everything to MongoDB.
- Backfill: a one-time job that pulled 45 days of past data (1,080 hourly rows) so there was something to train the first model on.
- Training Pipeline (daily): pulls everything from MongoDB, tests a few models against each other, and saves the best one back into the project's GitHub repo.
- Dashboard (on demand): loads whatever model was last saved and shows a live 72-hour forecast.

Two GitHub Actions workflows keep it all running: `hourly-features.yml` (every hour, at 5 past) and `daily-training.yml` (once a day, at 2am UTC). Both use GitHub's built-in secrets storage for the MongoDB password and API key, so nothing sensitive sits in the code itself.

3. Data Collection and Feature Engineering

3.1 Backfilling history

Before any model could be trained, there needed to be some history to learn from. A 45-day backfill (July 13 to August 27, 2026) pulled in 1,080 hourly rows by matching pollution readings with weather readings for the same hour.

3.2 What the model is actually predicting

AQI is calculated from PM2.5 using the official EPA formula, which converts a pollutant concentration into the familiar 0 to 500 scale. The thing being predicted, `target_aqi_72h`, is simply what the AQI will be 72 hours after each row's timestamp. So this is a genuine forecast, not just a snapshot of current conditions.

3.3 The 32 features that go into the model

- Raw pollutants: CO, NO, NO2, O3, SO2, PM2.5, PM10, NH3
- Weather: temperature, humidity, wind speed and direction, cloud cover, pressure
- Lag features: what AQI and PM2.5 were 24, 48, and 72 hours earlier
- Rolling stats: the 72-hour rolling average and standard deviation of AQI, plus a rolling average of PM2.5
- Cyclical time features: hour and day of week, encoded so the model understands that hour 23 is right next to hour 0, not far away from it
- Calendar features: hour, day, month, day of week

After dropping rows that did not have a full 72 hours of history behind them, or a valid target ahead of them, 936 clean rows were left to actually train on.

3.4 A data bug that was found and fixed

While checking the data, one row stood out: PM2.5 of 12.04 micrograms per cubic meter paired with an AQI of 500, the maximum possible (hazardous) reading. That did not make sense, since PM2.5 around 12 should give an AQI close to 50. Tracing it back, the EPA breakpoint table used to convert PM2.5 into AQI had a gap: it covered 0.0 to 12.0, then jumped to 12.1 to 35.4, with nothing covering values like 12.04 in between. Any reading that landed in that gap fell through to a fallback value of 500. The fix was simple: round PM2.5 to one decimal place before looking it up, which is actually how the EPA table is meant to be used. This mattered more than it might sound, because the 72-hour target is just a shifted copy of AQI, so that one bad value did not just corrupt its own row. It became the training target for a different row 72 hours earlier, quietly dragging down the model's accuracy until it was caught.

4. Modeling and Evaluation

4.1 The three models tried

- Ridge Regression: a simple linear baseline, with features scaled and alpha set to 10
- Random Forest: 300 trees, max depth of 8
- XGBoost: 200 trees, max depth of 4, learning rate 0.05

4.2 How the models were tested

The first attempt used a single 80/20 split: train on the first 80 percent of the data chronologically, test on the last 20 percent. That gave oddly bad results, negative R2 for every single model. It turned out the test week just happened to be calmer than the training weeks, since there is only 45 days of data total and one unlucky week can throw everything off. To get a trustworthy answer, the evaluation was redone with 5-fold time-series cross-validation: train on an expanding window, test on the next chunk, repeat five times, and average the results. This is the standard way to evaluate time-series models properly, and it avoids the problem of one lucky or unlucky test period skewing everything.

Two simple baselines were used as sanity checks, since a model is only useful if it beats these:

- Mean baseline: just predict the training average every time. RMSE = 10.39
- Persistence baseline: predict that AQI in 72 hours will be the same as it is right now. RMSE = 10.90

4.3 Results

Model	RMSE	MAE	R2	vs. Baseline
XGBoost	7.66	6.40	-1.63	Beats baseline
Random Forest	7.81	6.24	-1.67	Beats baseline
Ridge Regression	14.88	11.95	-10.46	Worse than baseline

XGBoost came out on top, with the lowest RMSE averaged across all five folds, so it is the model used in the final deployed system. Both tree-based models clearly beat the naive baselines (7.66 to 7.81 versus 10.39 to 10.90), which means they are picking up on real patterns, not just getting lucky. Ridge did worse than both baselines, which makes sense: the relationship between weather, pollutants, and AQI three days out is probably not a straight line, so a linear model like Ridge struggles with it.

One thing worth explaining: R2 is negative for every model, even though RMSE clearly beats the baselines. That is not a contradiction. Several of the test folds happen to cover calm weeks with very little variation in AQI, and R2 measures error relative to how much the target actually varies. When there is barely any variation to explain, R2 punishes even small errors harshly. RMSE and MAE, compared directly to the baselines above, give a much more honest picture of how well the model actually performs here.

5. Model Explainability (SHAP)

SHAP was used to look inside the final XGBoost model and see which features actually drive its predictions, and in which direction.

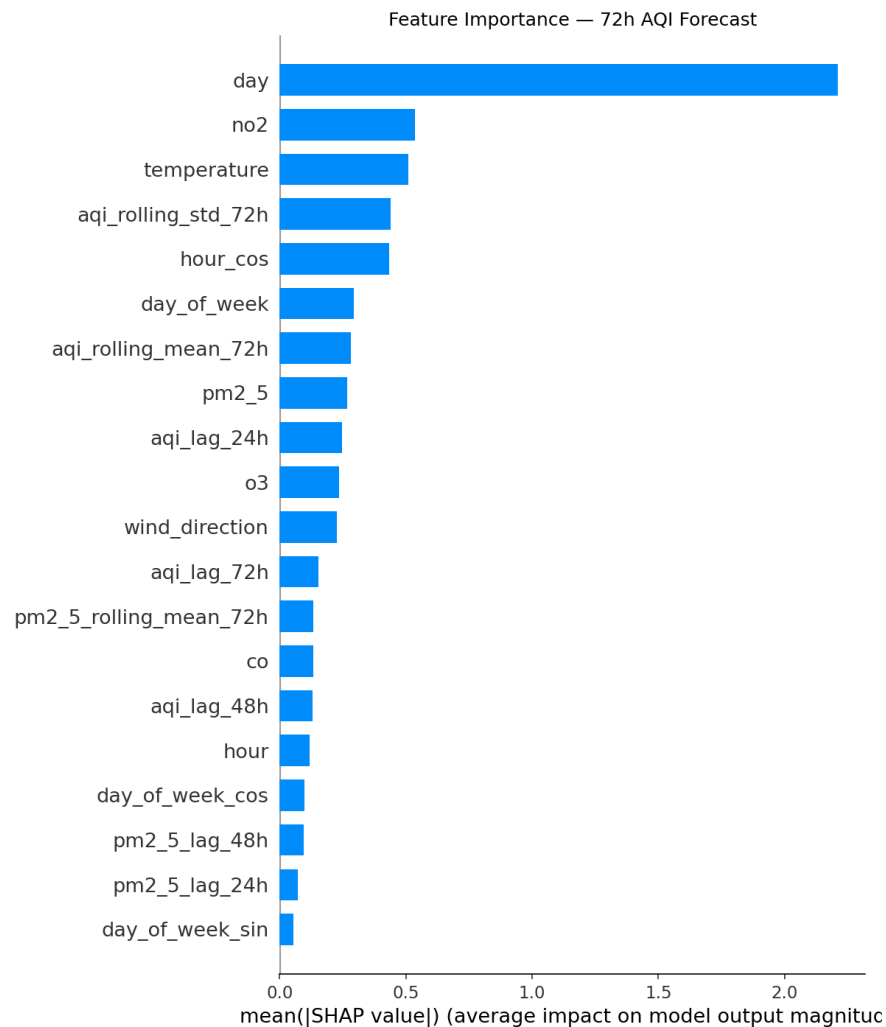


Figure 1: Average impact of each feature on the model's predictions, ranked.

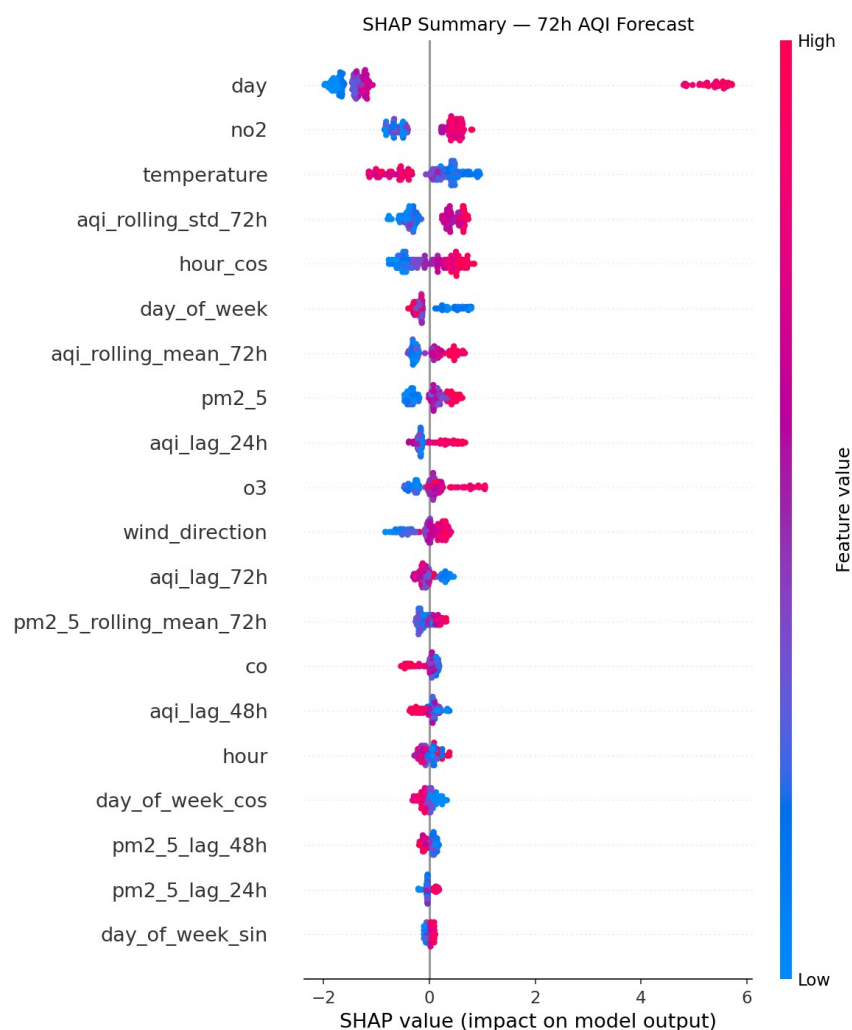


Figure 2: SHAP summary plot. Color shows whether the feature's value was high (red) or low (blue); position shows whether it pushed the prediction up or down.

5.1 A limitation worth being upfront about: the day feature

By far the most influential feature turned out to be day, the calendar day of the month, with an average impact more than four times higher than the next feature (no2). That is flagged here honestly rather than quietly accepted, because it is probably not measuring what it looks like it is measuring. The training data only covers a single 45-day window, mid-July to late August 2026. So the model most likely learned day as a stand-in for how far into the data collection period a row falls, rather than any real repeating pattern. Karachi's air quality does not actually depend on whether the date is the 5th or the 25th in a way that would hold up in a different month or year. As the automated pipeline keeps collecting data across more months and seasons, this dependency should naturally fade.

Leaving day aside, the next most important features make a lot more sense: no2 and pm2_5 (direct pollution readings), temperature (affects how pollution disperses in the air), the rolling AQI average and volatility (recent trend), and hour_cos (captures the daily rise and fall of traffic-driven pollution). These line up with what you would expect to actually drive short-term air quality.

6. Automation and Deployment

6.1 The two GitHub Actions workflows

- `hourly-features.yml`: runs every hour at 5 past, fetching new data and recalculating lag and rolling features using recent history before saving to MongoDB.
- `daily-training.yml`: runs once a day at 2am UTC, retrains all three models, and automatically commits the updated model file back to the repo.

Both workflows log in using GitHub's repository secrets (MONGODB_URI, OPENWEATHER_API_KEY) instead of hardcoded credentials, and both were tested by hand and confirmed working during development.

6.2 The dashboard

A Streamlit dashboard loads whatever model was most recently saved and the latest row of data from MongoDB to show a live forecast: the current AQI, the 72-hour prediction with how much it is expected to change, a hazard alert that adjusts based on EPA severity levels, and a chart of the last 72 hours.

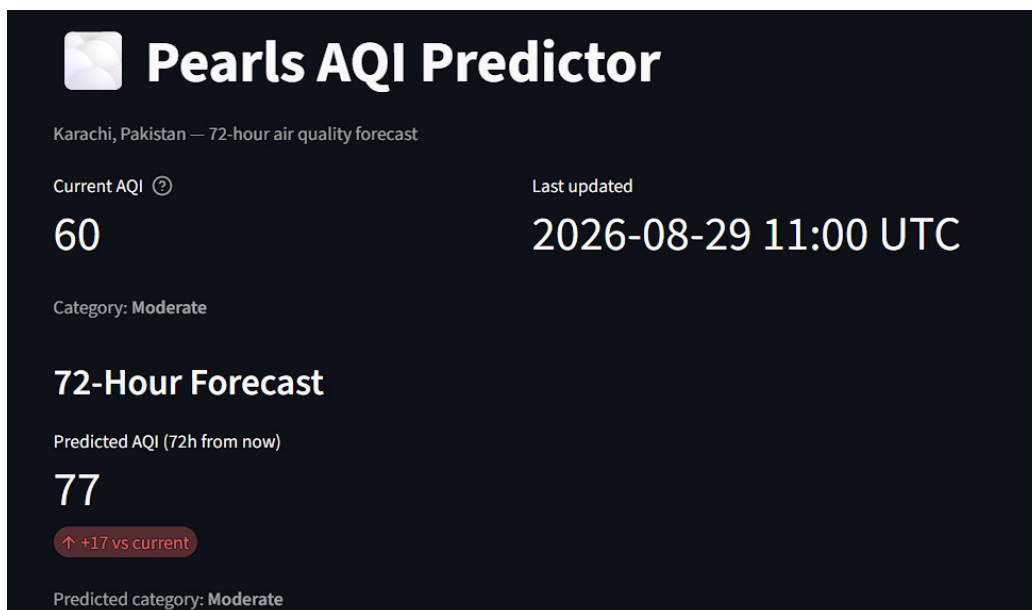


Figure 3: Dashboard after fixing the hourly lag-feature bug. Current AQI is 60 (Moderate), the 72-hour forecast is 77 (Moderate), and there is no data-quality warning.

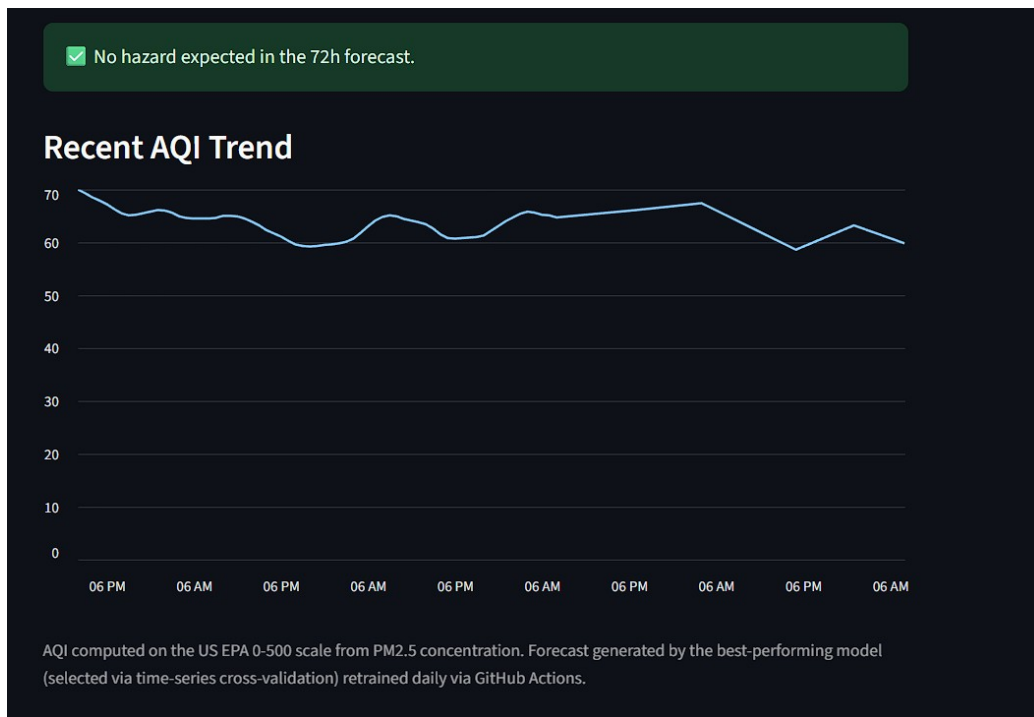


Figure 4: The hazard alert and the 72-hour AQI trend chart.

7. Challenges Along the Way

7.1 A silent bug in the database updates

The function meant to save new rows to MongoDB was originally using `insert_one()` wrapped in a `try/except` that quietly skipped any row that already existed. That sounds harmless, but it meant that if a row had already been saved with bad or incomplete data, it could never be corrected. It would just silently stay wrong forever. This was fixed by switching to `replace_one(upsert=True)`, so re-running the pipeline always overwrites old data with the latest, correct version.

7.2 The hourly job was not computing features correctly

The hourly collection job was calculating lag and rolling features using only the single new row it had just fetched, which meant those features came out blank every single time. This only became obvious after noticing a warning on the dashboard about missing data. The fix was to pull the last 80 hours of history from MongoDB first, then calculate the features properly, the same way the backfill script already did.

7.3 One test split was not enough

A single chronological train/test split turned out to be very sensitive to which specific weeks ended up in the test set, given only 45 days of data. It made even models that were learning real patterns look useless, since R^2 went negative across the board. Switching to 5-fold time-series cross-validation, and averaging across all five folds, gave a much steadier and more trustworthy result, and that is the number reported as this project's real result.

7.4 A gap in the EPA lookup table

The EPA AQI standard assumes $PM_{2.5}$ is rounded to one decimal place before looking it up in the breakpoint table. That rounding step was missed at first, which left a small gap between adjacent breakpoints. Because the 72-hour target is just a shifted copy of AQI, a single bad value like this does not just affect its own row, it becomes someone else's training target too. It is a good reminder to sanity-check formulas like this early, especially in any pipeline where values get shifted or reused later on.

8. What Could Come Next

- Let the automated pipeline keep running for a few more months to capture real seasonal patterns, which should reduce how much the model leans on the day feature
- Add confidence intervals to the forecast, so the dashboard shows a likely range instead of just one number
- Add automatic data-quality checks to the pipeline, like the AQI/PM2.5 consistency check that caught the earlier bug, so similar issues get caught right away next time
- Deploy the dashboard publicly on Streamlit Community Cloud so it is reachable from anywhere
- Try an LSTM or other sequence model once there is enough historical data to make that worthwhile

9. Conclusion

This project set out to build a fully automated system that forecasts Karachi's air quality three days ahead, from raw data collection all the way through to a live dashboard, and it does that. The final model, XGBoost, gets an RMSE of 7.66 through proper cross-validation, clearly beating both a naive average baseline (10.39) and a persistence baseline (10.90). Along the way, three real bugs got caught and fixed: a gap in the EPA breakpoint table, a database update that was silently failing, and an hourly job that was not computing features correctly. Each of those is written up here honestly, not glossed over, because finding and fixing problems like this is a normal and important part of building something that actually works. The SHAP analysis also points out a real limitation, the day feature likely not generalizing beyond this 45-day window, rather than pretending the model is a perfect black box. As the hourly pipeline keeps collecting more data over time, that limitation should naturally shrink on its own.